
Advance Machine Learning

Assignment 1

Shiva Shrestha

Department of Computer Science
Kennesaw State University
Assignment GitHub Repository

1 Introduction

In this assignment, as instructed by the professor, I have developed an agentic summarization system that processes PDF documents, divides their text into smaller chunks, converts those chunks into numerical embeddings, and finally groups similar chunks using my implementation of the K-Means algorithm.

A representative chunk from each cluster is used to generate a topic label and a short topic summary. At last, the topic summaries are combined together to produce a coherent summary of the complete document collection. The system is evaluated using different values of k , specifically $k = 3$, $k = 10$, and $k = 30$.

The remaining of the assignment is arranged as follows: Section 2 describes the architecture in brief, Section 3 describes the experimental configuration, Section 4 describes the evaluation metrics used for the evaluation of cluster, Section 5 explains the dataset, Section 6 explains the result, followed by Section 7 which contains the discussion of all of the questions asked in the assignments, and finally Section 8 concludes the assignment with Appendix A explaining the sections where AI were used.

2 System Architecture

The system is organized as a sequence of collaborating agents using LangGraph. Each agent performs one specific task and passes its output to the next agent through a shared state. The workflow follows this order:

1. First of all, the document agent extracts and divides PDF text into small 500 word chunks.
2. The embedding agent then converts each of those chunk into a numerical vector.
3. The clustering agent groups the embeddings using K-Means clustering algorithm.
4. The topic agent generates a label and summary for each cluster.
5. The summary agent combines the topic summaries into one final summary.

In my implementation, the embeddings are not persistently stored in any database, rather they are just stored as variables within the notebook implementation.

3 Experimental Configuration

The language model and embedding model were accessed locally through Ollama. All other experimental setup configurations are mentioned in the Tab. 1.

Table 1: Main experimental settings.

Setting	Value
Embedding model	embeddinggemma
Language model	gemma4:e2b
Agent orchestration	LangGraph
Clustering algorithm	Custom K-means
Distance metric	Euclidean
Number of clusters (k)	3, 10, 30
Chunk size	500 characters
Chunk overlap	200 characters
Hardware	MacBook M1 Pro(16GB RAM)

4 Evaluation Metrics

Clustering quality was assessed using inertia and the silhouette score. Inertia measures within-cluster compactness, while the silhouette score evaluates cohesion within clusters relative to separation from other clusters.

Inertia Inertia is the sum of squared Euclidean distances between each chunk embedding and its assigned cluster centroid:

$$J = \sum_{i=1}^n \|\mathbf{x}_i - \boldsymbol{\mu}_{c_i}\|_2^2, \quad (1)$$

where $\mathbf{x}_i$ denotes the embedding of chunk i , c_i identifies its assigned cluster, $\boldsymbol{\mu}_{c_i}$ is the corresponding centroid, and n is the total number of chunks. Lower inertia indicates greater within-cluster compactness. However, inertia generally decreases as the number of clusters k increases, so it was considered alongside the silhouette score when comparing clustering configurations.

Silhouette Score The silhouette coefficient evaluates how closely a chunk matches its assigned cluster relative to the nearest alternative cluster. For chunk i , it is defined as

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \quad (2)$$

where $a(i)$ is the average Euclidean distance to the other chunks in the same cluster, and $b(i)$ is the smallest average Euclidean distance to the chunks in any other cluster. The overall silhouette score is the mean coefficient across all chunks:

$$S = \frac{1}{n} \sum_{i=1}^n s(i). \quad (3)$$

For singleton clusters, the silhouette coefficient is set to zero; it is also set to zero when both $a(i)$ and $b(i)$ are zero. The score ranges from -1 to 1 . Higher values indicate compact, well-separated clusters, while values near zero indicate chunks near cluster boundaries. Negative values suggest that a chunk is, on average, closer to another cluster than to its assigned cluster.

5 Dataset

The dataset contains two collections with a total of seven research papers.

5.1 Transformer Collection

1. *Attention Is All You Need* [Vaswani et al. [2023]]
2. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding* (Devlin et al. [2019])

3. *LoRA: Low-Rank Adaptation of Large Language Models* [Hu et al. [2021]]
4. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity* [Fedus et al. [2022]]

5.2 Agentic System Security Collection

1. *Prompt Injection Attack Against LLM-Integrated Applications* [Liu et al., 2025]
2. *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents* [Debenedetti et al., 2024]
3. *A Cross-Layer Security Survey of LLM Agentic Systems: Attacks, Defenses, and Open Challenges* [SHARIF et al., 2026] (Preprint from our lab)

6 Results

6.1 Clustering Performance

Table 2: K-Means results for each document collection

Collection	k	Inertia	Silhouette Score
Transformers	3	417.7458	0.0465
Transformers	10	368.9357	0.0478
Transformers	30	322.2284	0.0507
LLM Security	3	385.3758	0.0699
LLM Security	10	339.1440	0.0615
LLM Security	30	296.5306	0.0529

For the Transformers collection, $k = 30$ produced the highest silhouette score of 0.0507. For the Agentic System Security Collection, $k = 3$ produced the highest silhouette score of 0.0699. Inertia decreased as k increased for both collections, which is as expected as additional clusters allow the data points to be located closer to their assigned centroids.

6.2 Transformer Collection

6.2.1 Results for $k = 3$

Table 3: Cluster medoids and generated topic labels for the Transformer collection with $k = 3$

Cluster	Size	Medoid Paper	Page	Chunk	Topic Label
0	260	<i>LoRA: Low-Rank Adaptation of Large Language Models</i>	7	3	NLP Model Efficiency
1	163	<i>Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity</i>	14	2	Transformer Model Efficiency
2	335	<i>LoRA: Low-Rank Adaptation of Large Language Models</i>	5	2	Transformer Optimization

All of the four papers in the Transformer collection were included in the clustering process. As $k = 3$, three cluster medoids were selected. Two medoids originated from different chunks of the LoRA paper, whereas the remaining medoid originated from the Switch Transformers paper.

The three clusters were labeled NLP model efficiency, efficient Transformer scaling, and Transformer optimization. Cluster 2 was the largest, containing 335 chunks, and Cluster 1 was the smallest, containing 163 chunks.

Generated Final Summary:

NLP model efficiency is addressed through various optimization techniques targeting parameter count, training methods, and runtime performance in Transformer models.

One approach focuses on optimizing the internal structure of Transformer models. By freezing MLP modules and adapting attention weights, it is possible to improve simplicity, parameter-efficiency, and reduce memory usage.

Other methods focus on optimizing the training process and scaling. For instance, optimizing pre-training recipes, such as RoBERTa optimizing BERT’s recipe, can boost task performance without introducing many trainable parameters, although this is challenged by larger models. Furthermore, model efficiency can be improved in terms of speed and sample efficiency; the Switch-based model demonstrates superior wall-clock speedup and sample efficiency compared to standard models while validating improved scaling properties.

6.2.2 Results for $k = 10$

Table 4: Cluster medoids and generated topic labels for the Transformer collection with $k = 10$

Cluster	Size	Medoid Paper	Page	Chunk	Topic Label
0	134	<i>LoRA: Low-Rank Adaptation of Large Language Models</i>	9	4	Parameter-Efficient Adaptation
1	70	<i>Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity</i>	35	4	Expert Routing and Tensor Operations
2	84	<i>Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity</i>	14	2	Transformer Model Efficiency
3	76	<i>BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding</i>	2	0	Bidirectional Pre-training
4	75	<i>BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding</i>	6	1	NLP Benchmark Results
5	85	<i>LoRA: Low-Rank Adaptation of Large Language Models</i>	1	2	Parameter-Efficient Fine-Tuning
6	58	<i>Attention Is All You Need</i>	6	1	Positional Encoding
7	44	<i>Attention Is All You Need</i>	10	4	Transformer Model
8	71	<i>Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity</i>	3	2	Sparse Expert Models
9	61	<i>BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding</i>	2	2	BERT Representations

All four papers were represented by at least one medoid when $k = 10$. Cluster 0 was the largest with 134 chunks, while Cluster 7 was the smallest with 44 chunks. Compared with $k = 3$, the additional clusters produced more specific topics, including positional encoding, bidirectional pre-training, parameter-efficient fine-tuning, sparse expert models, and expert routing.

Generated Final Summary:

The Transformer model is a sequence transduction architecture that utilizes self-attention mechanisms to replace recurrent layers, enabling significantly faster training in tasks such as translation. To inject necessary sequence information into models lacking recurrence or convolution, positional encodings are added to provide information about the relative or absolute position of tokens in a sequence.

Bidirectional pre-training methods, such as those used by BERT, train deep bidirectional representations by employing masked language modeling and next sentence prediction. This approach reduces the need for heavily-engineered architectures and allows models to achieve state-of-the-art performance in Natural Language Processing (NLP).

The efficiency of large models has been addressed through various architectural and fine-tuning techniques. Parameter-Efficient Fine-Tuning (PEFT), exemplified by Low-Rank Adaptation (LoRA), significantly reduces the number of trainable parameters when fine-tuning large Transformer models by imposing low-rank constraints on weight updates using bottleneck structures.

Further computational efficiency is achieved through sparse expert models, such as the Switch Transformer, which are designed to be sparsely activated experts. In complex models, expert routing and tensor operations are used to combine expert outputs and dispatch tokens based on a router mechanism, which requires specific mathematical steps and tensor manipulations.

Switch-based Transformer models have demonstrated superior sample efficiency and speed compared to standard dense scaling methods. Finally, performance is evaluated using NLP benchmarks, where results, including GLUE test scores and F1 scores, are compared against official scores to assess language model performance.

6.2.3 Results for $k = 30$

The paper names in Table 5 are shortened for readability. LoRA, BERT, and Switch Transformers refer to their complete paper titles listed in the dataset section (Section 5).

Table 5: Cluster medoids and generated topics for the Transformer collection with $k = 30$

Cluster	Size	Medoid Paper	Page	Chunk	Topic Label
0	28	<i>Attention Is All You Need</i>	1	4	Machine Translation and Transformer Generalization
1	22	<i>LoRA</i>	2	1	Low-Rank Adaptation
2	17	<i>LoRA</i>	12	1	Matrix Adaptation Analysis
3	35	<i>Switch Transformers</i>	6	6	Model Efficiency and Sparse Routing
4	8	<i>BERT</i>	6	9	Question Answering Modeling
5	14	<i>LoRA</i>	1	7	Fine-Tuning Large Models
6	27	<i>Switch Transformers</i>	21	1	Parallelism Strategies
7	9	<i>LoRA</i>	1	9	LLM Optimization
8	42	<i>BERT</i>	8	2	BERT Pre-training Tasks
9	17	<i>Attention Is All You Need</i>	9	7	Sequence-to-Sequence Modeling
10	13	<i>BERT</i>	13	1	NLP Model Architectures
11	10	<i>Switch Transformers</i>	15	6	Model Evaluation
12	27	<i>Switch Transformers</i>	33	5	Machine Learning Loss
13	54	<i>LoRA</i>	1	2	Low-Rank Adaptation
14	22	<i>BERT</i>	4	2	Sequence Representation
15	41	<i>BERT</i>	6	1	Language Model Evaluation
16	17	<i>BERT</i>	1	1	BERT
17	19	<i>Switch Transformers</i>	11	2	Fine-Tuning Regularization Results
18	28	<i>Switch Transformers</i>	35	4	Mixture-of-Experts Routing
19	30	<i>LoRA</i>	3	6	Adapter Layer Latency
20	43	<i>Switch Transformers</i>	3	2	Switch Transformer
21	22	<i>Switch Transformers</i>	4	6	Model Scaling and Efficiency
22	50	<i>Switch Transformers</i>	14	2	Transformer Model Efficiency
23	55	<i>Attention Is All You Need</i>	5	3	Attention Mechanisms
24	22	<i>Switch Transformers</i>	10	5	Initialization Scale in Training
25	31	<i>Switch Transformers</i>	1	3	Model Training Efficiency
26	17	<i>BERT</i>	4	10	Masked Language Modeling
27	15	<i>BERT</i>	1	8	Language Model Training
28	13	<i>BERT</i>	8	4	Model Pre-training and NSP Impact
29	10	<i>Attention Is All You Need</i>	1	8	Machine Learning Implementation

All four papers were represented among the medoids for $k = 30$. Cluster 23 was the largest with 55 chunks, while Cluster 4 was the smallest with 8 chunks. The larger value of k produced more specific topics than $k = 3$ and $k = 10$, but several topics were closely related or repeated. Examples include multiple clusters about LoRA, BERT pre-training, and Switch Transformer efficiency.

Generated Final Summary:

Machine Translation and Transformer Generalization

The foundation of modern NLP is built upon the Transformer model, which has demonstrated strong generalization ability across various Natural Language Processing (NLP) tasks, including state-of-the-art performance in machine translation. The mechanism relies on attention, which governs the

flow of queries, keys, and values in encoder-decoder and self-attention layers to facilitate sequence-to-sequence modeling. The implementation of these models has been significantly advanced through optimized codebases, such as the tensor2tensor codebase, which accelerated research and results.

Language Model Foundations and Pre-training

BERT is a key pre-trained model designed to learn deep bidirectional representations by jointly conditioning on both left and right context, enabling state-of-the-art fine-tuning for various tasks. The effectiveness of these models is evaluated through comparative test results and F1 scores on benchmarks like GLUE, including data for models such as BERT and GPT.

The training of these language models involves various objectives. Masked Language Modeling (MLM) involves predicting masked words during training, and the importance of deep bidirectionality is evaluated by comparing models trained with different pre-training objectives, such as Masked LM and Left-to-Right (LTR) tasks. Furthermore, the impact of tasks like Next Sentence Prediction (NSP) on model performance is examined, as is the structuring and differentiation of sentence pairs within a sequence for classification.

Model Scaling and Efficiency Challenges

Fine-tuning large pre-trained models presents critical deployment challenges due to the massive number of parameters, necessitating the search for mitigation strategies. The power-law scaling of large models suggests that training large models on small datasets is the computationally optimal approach.

To manage the efficiency of large models, research focuses on optimizing training and inference. Methods for partitioning model weights and data tensors across cores are used in model and data parallelism strategies. Furthermore, the study of Model Scaling and Efficiency demonstrates that Transformer models utilizing Switch layers are more sample efficient and faster than standard dense scaling methods while maintaining superior scaling properties.

Optimization Techniques and Adaptation

To improve operational efficiency for pre-trained models, techniques focus on reducing parameter count and improving routing. Low-Rank Adaptation (LoRA) is a technique that freezes pre-trained model weights and injects trainable matrices, significantly reducing the number of parameters required for fine-tuning large language models. This approach is motivated by leveraging the low intrinsic rank of weight changes in over-parametrized models.

The introduction of Adapter layers, while useful, introduces inference latency in large-scale production scenarios, prompting research into variants and methods for latency reduction.

Model efficiency is further enhanced through sparse routing and specialized architectures. The Switch Transformer is proposed as a sparsely-activated expert model designed to achieve greater computational efficiency. This is supported by methods for efficient sparse routing, such as Mesh-TensorFlow (MTF), which addresses expert overflow by managing batch size and capacity factor. The system requires specific tensor operations and layout to combine expert outputs, route tokens, and calculate expert inputs.

Specific Applications and Mechanisms

Specific NLP tasks utilize specialized modeling techniques. Question Answering modeling employs methods for predicting the answer text span using packed sequences and start/end vectors. Matrix Adaptation Analysis describes a method using singular value decomposition to project weight matrices, thereby understanding the underlying mechanism of adapting pre-trained language models.

The calculation of machine learning loss, such as the load balance loss for Switch Transformers, can be performed using methods like Mesh Tensor-flow.

6.3 LLM Security Collection

6.3.1 Results for $k = 3$

All three security papers were included in the clustering process. However, only the Cross-Layer Security Survey and Prompt Injection papers supplied medoids for $k = 3$. No chunk from the AgentDojo paper was the closest chunk to any one of the three centroids.

Table 6: Cluster medoids and generated topic labels for the LLM Security collection with $k = 3$

Cluster	Size	Medoid Paper	Page	Chunk	Topic Label
0	389	<i>A Cross-Layer Security Survey of LLM Agentic Systems: Attacks, Defenses, and Open Challenges</i>	1	3	LLM Agent Security
1	50	<i>A Cross-Layer Security Survey of LLM Agentic Systems: Attacks, Defenses, and Open Challenges</i>	17	13	Academic Citation
2	297	<i>Prompt Injection Attack Against LLM-Integrated Applications</i>	1	15	Prompt Injection Attacks

Cluster 0 was the largest with 389 chunks, followed by Cluster 2 with 297 chunks, while Cluster 1 contained just 50 chunks. Its medoid consisted primarily of publication information and produced the label “Academic Citation.” **The main reason for this issue was that the PDF extractor treated the document’s header and footer as regular text. For example, the footer “ACM Comput. Surv., Vol. 1, No. 1, Article—Publication date: September 2026” became a separate text chunk. Consequently, it was embedded and included in the clustering process, producing a cluster with mediod labeled “Academic Citation.”**

Generated Final Summary:

The security of LLM-based agent systems requires a comprehensive approach, which includes establishing a unified threat model and a five-layer deployment stack for addressing security threats and implementing defenses.

A critical area of concern involves Prompt Injection Attacks. Successful prompt attacks on LLMs occur when malicious payloads are used to trick the model into interpreting them as legitimate questions, drawing inspiration from traditional injection attacks.

6.3.2 Results for $k = 10$

Table 7: Cluster medoids and generated topic labels for the LLM Security collection with $k = 10$

Cluster	Size	Medoid Paper	Page	Chunk	Topic Label
0	70	<i>AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents</i>	2	3	Agent Evaluation
1	130	<i>A Cross-Layer Security Survey of LLM Agentic Systems</i>	5	8	LLM Agent Security Model
2	122	<i>A Cross-Layer Security Survey of LLM Agentic Systems</i>	17	0	LLM Agent Security
3	49	<i>Prompt Injection Attack Against LLM-Integrated Applications</i>	8	18	LLM Prompt Generation and Attack Components
4	149	<i>Prompt Injection Attack Against LLM-Integrated Applications</i>	1	15	Prompt Injection Attacks
5	49	<i>A Cross-Layer Security Survey of LLM Agentic Systems</i>	4	7	LLM Agents
6	41	<i>A Cross-Layer Security Survey of LLM Agentic Systems</i>	18	9	AI Security Threats
7	23	<i>Prompt Injection Attack Against LLM-Integrated Applications</i>	4	16	AI Prompt Injection Attacks
8	47	<i>A Cross-Layer Security Survey of LLM Agentic Systems</i>	17	13	Journal Citation
9	56	<i>Prompt Injection Attack Against LLM-Integrated Applications</i>	6	6	Prompt Injection Attack

All three papers were represented by at least one medoid for $k = 10$. Cluster 4 was the largest with 149 chunks, where as Cluster 7 was the smallest with 23 chunks. Compared with $k = 3$, this configuration identified more specific topics. However, several prompt-injection topics overlapped,

and Cluster 8 again represented publication metadata instead of substantive paper content due to same reason as mentioned for $k = 3$.

Generated Final Summary:

Multi-Document Summary: LLM Agent Security and Prompt Injection

LLM Agents and Security Frameworks

LLM-based agents are systems that combine a core Large Language Model with capabilities for planning, tool use, and memory, enabling them to orchestrate multi-step task execution and external actions. To systematically analyze the security of these systems, a five-layer deployment stack model is introduced, which organizes components and identifies where security assumptions change.

Current multi-agent trust frameworks remain underdeveloped, and severe agent-specific threats, such as prompt injection and memory poisoning, concentrate in higher layers of AI systems, exposing significant gaps in current defenses. Attacks like planning subversion specifically target agent integrity and safety properties at higher layers of the system.

Prompt Injection Attacks

Prompt injection attacks exploit the mechanism by which LLMs process input. Successful attacks hinge on tricking the model into interpreting malicious payloads as questions rather than data, drawing inspiration from traditional injection attacks. Researchers have proposed novel methodologies, such as HOUYI, tailored for prompt injection attacks in black-box scenarios involving LLM-integrated applications.

To facilitate testing and exploitation, methods exist for generating prompt components, including Separators and Disruptors, and managing injection failures during an attack scenario. These techniques are used to construct payloads designed to test and exploit AI systems.

Agent Evaluation

To assess the security and utility of AI agents, methods like AgentDojo evaluate agents and attackers by using formal utility checks over the environment state. This approach is designed to accurately reflect the utility-security tradeoff in adversarial settings.

6.3.3 Results for $k = 30$

The paper titles are shortened in Table 8 for readability. Their complete titles are provided in the dataset section(Section 5).

Table 8: Cluster medoids and generated topics for the LLM Security collection with $k = 30$

Cluster	Size	Medoid Paper	Page	Chunk	Topic Label
0	7	<i>Prompt Injection Attack</i>	5	5	LLM Input Handling
1	42	<i>AgentDojo</i>	2	4	Agent Security Evaluation
2	27	<i>Cross-Layer Security Survey</i>	6	3	Agent System Security and Architecture
3	17	<i>Cross-Layer Security Survey</i>	12	8	Attack Taxonomy
4	22	<i>AgentDojo</i>	8	8	Prompt Injection Defenses
5	3	<i>AgentDojo</i>	5	0	Agent Task Definition
6	43	<i>Cross-Layer Security Survey</i>	3	6	Defense Evaluation and Evolution
7	2	<i>AgentDojo</i>	4	5	Calendar Event Retrieval
8	1	<i>Prompt Injection Attack</i>	11	16	AI Security Vulnerabilities
9	15	<i>Prompt Injection Attack</i>	5	1	Prompt Injection Security
10	37	<i>AgentDojo</i>	2	11	LLM Capabilities and Security
11	24	<i>Cross-Layer Security Survey</i>	18	9	AI Agent Security
12	64	<i>Cross-Layer Security Survey</i>	5	8	LLM Agent Security Model
13	11	<i>AgentDojo</i>	4	1	AI Agent Environments
14	7	<i>Prompt Injection Attack</i>	10	2	AI Feature Matrix
15	53	<i>Prompt Injection Attack</i>	1	9	Prompt Injection
16	28	<i>Prompt Injection Attack</i>	7	2	Prompt Injection
17	37	<i>Cross-Layer Security Survey</i>	28	3	AI Security and Attestation

Continued on the next page

Table continued from the previous page

Cluster	Size	Medoid Paper	Page	Chunk	Topic Label
18	20	<i>Cross-Layer Security Survey</i>	14	5	LLM Injection Attacks
19	26	<i>AgentDojo</i>	9	7	Agent Security and Isolation
20	24	<i>AgentDojo</i>	9	1	Prompt Injection Defenses
21	60	<i>Cross-Layer Security Survey</i>	1	6	LLM Agent Security
22	22	<i>Cross-Layer Security Survey</i>	17	13	Academic Citation
23	6	<i>Prompt Injection Attack</i>	4	10	Decision Analysis and Security
24	38	<i>Cross-Layer Security Survey</i>	13	0	LLM Agent Security
25	7	<i>Cross-Layer Security Survey</i>	4	13	Date
26	54	<i>Prompt Injection Attack</i>	6	6	Prompt Injection Attacks
27	21	<i>Prompt Injection Attack</i>	8	0	LLM Framework Generation
28	5	<i>Prompt Injection Attack</i>	12	0	Prompt Leaking Security
29	13	<i>Prompt Injection Attack</i>	12	9	LLM Security and Prompt Abuse

All three papers were represented among the medoids for $k = 30$. Cluster 12 was the largest with 64 chunks, while Cluster 8 contained only one chunk. This configuration identified detailed concepts such as attack taxonomies, agent isolation, prompt leaking, prompt abuse, security attestation, and defense evaluation.

However, due to the header and footer of the paper same problem as $k = 3$ and $k = 10$ persist and over-amplified in $k = 30$, with nonsense topic labels like "Date" and "Academic Citations". The silhouette score also decreased from 0.0699 for $k = 3$ to 0.0529 for $k = 30$. Therefore, $k = 30$ provided greater topic detail but weaker overall cluster separation.

Generated Final Summary:

The security of Large Language Models (LLMs) and their integrated agent systems is critically dependent on how they handle user input and execute tasks. A primary security concern is prompt injection, where malicious users employ harmful prompts to override the original instructions of LLMs, posing a significant hazard in integrated applications. This vulnerability is amplified when LLMs are used in agent systems, where compromised model behavior can trigger harmful actions, such as erroneous tool calls, planning errors, or privacy leakage.

Prompt injection attacks can be executed through various channels, including hidden instructions in web pages, where external data can be injected to manipulate LLM-integrated applications and exfiltrate user data. Furthermore, specific methodologies, such as HOUYI, have been proposed for prompt injection tailored for LLM-integrated applications operating in black-box scenarios. Related concerns include prompt leaking security, which arises from the unintentional divulging of internal prompts when replicating LLM-integrated applications using leaked prompts, and prompt abuse, which creates security loopholes and exposes developers to financial risks through unintended usage.

To manage these risks, defenses against prompt injection are necessary. Research evaluates the utility-robustness and attack success rates of various defenses, such as data delimiters, against these attacks. Effective mitigation strategies involve implementing isolation techniques, such as using a planner to dispatch calls to isolated agents, to mitigate prompt injection risks when agents handle multiple tasks.

The security of LLM-based agents requires a structured architectural approach. A five-layer deployment stack model is introduced to systematically analyze the security of these systems by organizing components and identifying where security assumptions change. This framework provides a unified architectural basis for securing LLM-based agents and outlines directions for robust and trustworthy agentic systems.

Agent systems often utilize complex workflows, such as Retrieval-Augmented Generation (RAG) and vector databases, which involve assets and security concerns that must be addressed. An attack taxonomy is mapped onto the deployment stack to categorize these threats. Furthermore, the security evaluation process involves analyzing attack propagation, evaluating defenses against deployment criteria, and tracing the evolution of security defenses. Frameworks like AgentDojo are extensible tools designed to evaluate agents and attackers using formal utility checks over the environment state to address the evolving nature of ML security.

The interaction between LLMs and external environments is also a focus of security research. AI Agent Environments describe how to define a stateful workspace environment and enable an AI agent to interact with it using various tools. This includes defining specific agent tasks, such as instructing an agent to find and summarize calendar appointment details, and defining functions for specific actions, like retrieving calendar events by parsing date strings.

Overall, securing LLM agents involves understanding the capabilities and vulnerabilities of the models, analyzing the security landscape, and implementing layered defenses. Future work in this area focuses on lightweight tool attestation, runtime monitoring, and privacy mechanisms to ensure the trustworthiness of agentic systems.

7 Discussion

7.1 How Well Did Clustering Identify the Major Topics?

The K-Mean clustering implemented in this assignment was able to identify the main topics in both document collections. The Transformer papers covered topics such as self-attention, BERT, Low-Rank Adaptation (LoRA), positional encoding, and sparse expert models. The LLM Security papers contained the topics such as prompt injection, attacks, defenses, evaluation, and security architectures.

The results were clearest, and covered most of the topics with out any repetation with $k = 10$ as seen in Section 6. When I used $k = 3$, K-Mean clustering produced broad clusters, where as using the larger value of $k = 30$ created several small, repeated, and less meaningful clusters. The low silhouette scores of (0.0465–0.0699) suggest that many topics overlapped, which is reasonable because the papers within each of the group discussed closely related subjects.

7.2 Were Some Clusters Redundant?

When I set the value of k to 30, several clusters topic became redundant. We can clearly see transformer collection produced overlapping topics related to LoRA, BERT, as well as model optimization, while the LLM Security collection produced multiple cluster of same topic of prompt injection and agent security(Section 6). Some clusters captured publication metadata because PDF formatting caused headers and footers to be included in the extracted text and embeddings. This resulted in labels such as Academic Citation,” Journal Citation,” and “Date.” These results show that a larger k can create repetitive or uninformative clusters.

7.3 Did Increasing k Improve or Degrade Summary Quality?

When I tried to increase k , it improved the summaries only up to a certain point. With $k = 3$, the summaries were short but too broad, causing several paragraphs to be combined into a single board umbrella. At $k = 10$, the clusters provided better topic coverage while keeping the summaries clear and organized. Finally, when we set $k = 30$, it produced more detailed summaries, however many topics became repetitive or overly specific. Some clusters also represented header and footer metadata instead of meaningful content. Based on the results of Section 6, $k = 10$ provided the best balance of detail, coverage, and readability.

7.4 Were the Generated Topic Labels Accurate?

Most of the topic labels accurately represented their medoid chunks. Labels such as “Positional Encoding,” “Low-Rank Adaptation,” “Sparse Expert Models,” and “Prompt Injection Attacks” were consistent with the source content. However, labels such as “Date,” “Academic Citation,” and “Journal Citation” described header or footer metadata rather than meaningful topics, which was due to formatting errors.

The locally executed gemma4:e2b model produced useful labels but occasionally generated general or repetitive results, which is understandable for the sub 6 billion parameter models. This was more noticeable with $k = 30$, where over-fragmentation created many similar clusters and thus many repeated cluster topics. In sum, label quality generated in this assignment was affected by noisy

PDF text, the selected value of k , and the limitations of the sub 6 billion parameter size local model, thus some being pretty accurate topic label while some being completely unrelated.

7.5 What Challenges Arose When Ordering Summary Components?

The cluster numbers generated by using k-Means clustering of text embeddings/tokens doesn't represent a logical topic order. Therefore, combining summaries by cluster number generally placed different sections in random ways like evaluation before model architecture or defenses before the related attacks.

Although, the summary agent were specified to make the connection between related idea smooth and reorganize the topics into a more natural sequence, however this became harder with $k = 30$ because several components were repetitive, overly specific, or based on document metadata. As a result, the main challenges were the lack of an inherent cluster order, overlapping topics, noisy chunks, and the larger number of components that needed to be organized.

7.6 How Effective Was the Agent-Based Workflow?

The agent-based workflow which I implemented using LangGraph framework effectively divided the summarization process into document processing, embedding, clustering, medoid selection, topic generation, and the final summarization. LangGraph passed a shared state between these stages, making the pipeline easier to test and maintain. It also allowed the same chunks and embeddings to be reused for different values of k , thus allowing me to reduce the processing time.

However, the main limitation was the sequential execution. Each cluster required a separate Ollama request, making $k = 30$ experiments extremely slow in my local laptop. The workflow also lacked automatic mechanisms for removing noisy medoids (which was common in LLM Security Collection), combining redundant topics, or reviewing weak summaries. Overall, it worked well as a structured pipeline, but it was not a fully adaptive multi-agent system.

8 Conclusion

In this assignment, I successfully developed an LLM based agentic pipeline for summarizing multiple documents using embeddings, K-Means clustering, medoid selection, and locally hosted large language models. Among all of the tested configurations, $k = 10$ provided the best balance between topic coverage and readability. Although noisy PDF text and redundant clusters affected some results, the workflow effectively identified and summarized the main themes in both of the tested document collections.

A Generative AI Use Disclosure

OpenAI Codex assisted with debugging, code organization, whereas ChatGPT-5.6 Sol was involved with LaTeX formatting (Particularly formatting the cluster topics into the table) and improving written explanations along with Grammarly for grammar corrections. All suggestions were reviewed, tested, and revised by the author.

The system used local Ollama models: `embeddinggemma` generated document embeddings, while `gemma4:e2b` generated topic labels and summaries mentioned in the section 6. The author(I) implemented the custom K-Means algorithm (from scratch, although I took some inspiration from general NumPy-based procedure described by Sharma(Sharma [2022])), conducted the experiments, verified the results, and takes responsibility for the final submission.

References

Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents, 2024. URL <https://arxiv.org/abs/2406.13352>.

- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019. URL <https://arxiv.org/abs/1810.04805>.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, 2022. URL <https://arxiv.org/abs/2101.03961>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021. URL <https://arxiv.org/abs/2106.09685>.
- Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, Leo Yu Zhang, and Yang Liu. Prompt injection attack against llm-integrated applications, 2025. URL <https://arxiv.org/abs/2306.05499>.
- KAZI SHAHARAIR SHARIF, SHIVA SHRESTHA, ELISA BERTINO, JIADONG LOU, DAVID WU, and HONGHUI XU. A cross-layer security survey of llm agentic systems: Attacks, defenses, and open challenges. *ACM Comput. Surv*, 1(1), 2026.
- Sajal Sharma. Coding k-means clustering using python and numpy. DEV Community, September 2022. URL <https://dev.to/sajal2692/coding-k-means-clustering-using-python-and-numpy-fg1>. Accessed: September 17, 2026.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL <https://arxiv.org/abs/1706.03762>.